
Policy Gradient Methods For Overcooked Environment

Leonardo Chiarioni
Artificial Intelligence
Università di Bologna
leonardo.chiarioni@studio.unibo.it

Abstract

Agentic systems and reinforcement learning (RL) are hot topics in AI today; autonomous agents in an environment achieving user-defined objectives and a learning paradigm to teach such agents to interact with the environment through explorations and rewards based on the actions performed. Non-playable-character in many videogames are trained with reinforcement learning [Wang et al., 2010], chess players study games played by RL agents [Silver et al., 2017] and even Large Language Models (LLMs) are aligned with human and ethical preferences through reinforcement learning [Lambert, 2025], [Tennant et al., 2025]. Given the relevance and the applicability of such paradigm it is worth studying, making research and gaining knowledge on this field. In this report we explore learning algorithms to train RL agents in a *cooperative game environment*, overcooked, with the goal of making soups and deliver them to customers as fast as possible. We demonstrate how policy gradient methods are suitable approaches to solve the task at hand, how rewards shaping is necessary to improve the training time and avoid local optima (e.g. preparing soups without delivering them) and how hyper-parameters tuning is crucial to speed up the learning.

1 Introduction

Training RL agents can be as challenging as fascinating, designing algorithms that tell the agent what is good and what is bad, and then let it learn alone by exploring the environment, taking actions and observing what happens. Early attempts based on tabular methods [Watkins and Dayan, 1992] have proved that this approach is valuable and it works, even though it tends to be unfeasible as the number of possible configurations of the environment grows; value approximation and policy gradient methods [Mnih et al., 2015], Schulman et al. [2017] have overcome the complexity limit and have solved games that were previously thought to be unsolvable [Silver et al., 2016], thanks to the ability of neural networks (NNs) to approximate any function [Cybenko, 1989]. With our work we propose and illustrate two implementations of policy gradient methods, namely actor-critic (AC) and proximal-policy-approximation (PPO), together with reward strategies and hyperparameters settings, to train two reinforcement learning agents in overcooked environment, a standardized cooperative multi-agent environment designed to test out and compare different RL algorithms. We will describe the aspects in which we focused on, the experimental setups and we will discuss the results, highlighting critical points and potential improvements of the solution.

2 System description

2.1 Overcooked-ai

Overcooked-AI [Human Compatible AI Lab, 2020] is a benchmark environment for fully cooperative human-AI task performance, based on the wildly popular video game Overcooked. The goal of the game is to deliver soups as fast as possible by placing up to 3 ingredients in a pot, waiting for the soup to cook, and then having an agent pick up the soup and delivering it.

2.1.1 Layouts

Overcooked makes available different layouts in which to test the agents’ cooperative skills. Our work will focus on solving *cramped_room* layout.

2.1.2 Rewards

The rewards returned by the environment are defined in table 1

Table 1: Rewards of the environment

Reward	Type	Value
Potting onion	Shaped	3
Useful plate pickup	Shaped	3
Soup pickup	Shaped	5
Soup delivery (≥ 3 onions)	Sparse	20

2.1.3 Observation space

This is a fully-observable MDP and both agents have access to the full observation. The observations returned by the environment are vectors of 96 elements, one for each agent.

2.1.4 Action space

The action space is discrete with six possible actions: up, down, left, right, stay, and interact.

2.2 Deep learning framework

Since most of the implementations available online are based on Pytorch framework, we took an additional challenge and decided to implement our algorithms with Keras Tensorflow framework.

3 Backgrounds

Policy gradient methods are learning algorithms for finding the optimal policy (i.e. control problem) that select the action to perform without needing to consult a value function. The value function will still be needed to learn the policy, but it’s not involved in the exploration by the agent. Both the policy function π and value function v are approximated with NNs. We assume the reader to be familiar with the notation and such learning paradigm. In this section we will briefly introduce the mathematical formulation of two learning algorithms used as **inspiration** for our implementation.

3.1 Actor-critic

Our implementation of actor-critic, inspired by [Musolesi, 2025, slide 8], uses a value function approximation trained with the semi-gradient method, as follows:

$$w_{t+1} = w_t + \alpha^w \delta_t \nabla \hat{v}(S_t, w) \quad (1)$$

while the actor’s weights are updated with:

$$\theta_{t+1} = \theta_t + \alpha^\theta \delta_t \nabla \ln \pi(A_t | S_t, \theta) \quad (2)$$

For both, we have:

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, w) - \hat{v}(S_t, w) \quad (3)$$

3.2 Proximal policy approximation

Our implementation of PPO, inspired by [Huang et al., 2022], [Musolesi, 2025, slide 28], uses a value function approximation trained with the mean squared error (MSE) between the actual return and the estimated one, as follows:

$$w_{t+1} = w_t + \alpha^w \delta_t \nabla L_v(S_t) \quad (4)$$

with:

$$L_v(S_t) = \frac{1}{2} (v_t(S_t) - \hat{v}_t(S_t, w))^2 = \frac{1}{2} \delta_t^2 \quad (5)$$

while the actor’s weights are updated with the ppo clip:

$$\theta_{t+1} = \theta_t + \alpha^\theta \delta_t \nabla L_\pi(S_t, A_t, \theta_t, \theta) \quad (6)$$

with:

$$L_\pi(S_t, A_t, \theta_t, \theta) = \min\left(\frac{\pi_\theta(A_t|S_t)}{\pi_{\theta_t}(A_t|S_t)}, \text{clip}\left(\frac{\pi_\theta(A_t|S_t)}{\pi_{\theta_t}(A_t|S_t)}, 1 - \epsilon, 1 + \epsilon\right)\right) \quad (7)$$

For both, again we have (3) to hold.

4 Project goal and workflow

The goal of our project was to prove that policy gradient methods introduced in section 3 are suitable tools for solving the cooperative game environment, thus we focused on *cramped_room* layout. To do so, we followed Occam’s razor approach: we started with a simple solution, studied the results and iteratively tried to improve it by using more advanced algorithms and strategies to help the agent improve their policy.

4.1 Neural networks as function approximators

The neural networks used as approximators for the policy and the value functions are made of 3 linear layers, with 64, 128 and 64 neurons respectively, each of them with *tanh* activation function. They take as input the observation of each agent explained in section 2.1.3

4.1.1 Policy network

The final layer of the policy network outputs via *softmax* two probability distributions $\pi_t = (\pi_{1t}, \pi_{2t})$, one for each agent. The probability distributions over the six possible actions are the policies from which our agents will sample to determine their actions. The policy network is shared by the agents.

4.1.2 Value function network

For simplicity, the final layer of the value function network outputs a single value $\hat{v}_t$, shared among the two agents, corresponding to the state value associated to a specific state S_t . For this reason, the rewards R_t (and so the advantage function δ_t) are calculated by summing up the individual rewards obtained by the agents.

4.2 Reward strategies

In addition to the sparse rewards returned by the environment, we also used the shaped rewards shown in table 1, together with additional strategies to improve the agents’ policies.

4.2.1 Rewards to actions prior to reward

Inspired by how chess agents are trained, we found useful to reward also the previous K actions that led to a reward, rewarding a successful trajectory. Considering a generic timestep t in which a reward R_t has been delivered, we have:

$$\forall \tau \in [t - K; t - 1] : r_\tau = \gamma^{t-\tau} R_t \quad (8)$$

with r_τ being the custom shaped reward assigned at timestep τ .

Table 2: Shared hyperparameters across all the experiments

Hyperparameter	Description	Value
γ	Future value estimation and reward discount	0.95
ϵ_{ppo}	PPO clip value	0.05
α_{ac}^w	Learning rate value function for AC	1e-5
α_{ppo}^w	Learning rate value function for PPO	1e-4
T	Episode time limit	400
K	Previous action to reward	5
D_t	Delivery reward for incomplete soups	5

4.2.2 Rewards to incomplete soups

As shown in table 1, soups made of less than 3 onions are not rewarded when delivered; this might discourage the agents to learn how to deliver the soups early during training. For this reason we decided to assign a reward D_t when a soup is delivered at time t and a discounted reward d_τ to the K actions prior to t , according to:

$$\forall \tau \in [t - K; t] : d_\tau = \gamma^{t-\tau} D_t \quad (9)$$

4.3 Episodes rollout and training

We decided to use an hybrid approach between a monte-carlo and a temporal difference method for the episodes rollout; an entire episode is generated and training is performed only at the end, but the advantage function is computed at each timestep t of the episode according to (3). For this reason, the weights are actually updated after $T = 400$ time steps (episode duration), differently from (1), (2), (4) and (6). More specifically, we implemented stochastic gradient descent with $epochs = 2$ and $batch_size = 20$, for a total of $(T/batch_size) \times epochs = 40$ weight updates for each episode. In this way, we were able to speed up the training by using batches and potentially reduce the variance introduced by monte-carlo methods by using one-step estimations.

5 Experimental setup and training

We have set up different experiments by combining different policy gradient methods (section 3), hyperparameters and rewards strategies (section 4.2). We trained each experiment on 2000 episodes and used the average episodic reward (**sparse + shaped**) as a metric to evaluate the training process (figure 1), then we tested each of them on 10 episodes and reported the average statistics in table 5. We used the **shaped** reward as the final test metric. All the experiments were carried out by setting the *seed* = 42 to make the experiments reproducible. We also set some shared hyperparameters across all the experiments as reported in table 2.

5.1 Baseline

In order to understand whether our agents are actually able to learn something, we calculated the average episodic reward achieved by two random agents (i.e. agents that behave randomly) across 10 episodes and used it as a *baseline* (figure 1, table 5).

5.2 Actor-critic experiments

Our first approach involved the actor-critic algorithm presented in section 3.1. We set up 3 different experiments as shown in table 3, trained our agents and tested them.

5.3 Proximal-policy-approximation experiments

The second approach involved the PPO algorithm presented in section 3.2. We set up again 3 different experiments as shown in table 4, trained our agents and tested them.

Table 3: Actor-critic experiments’ setups

Experiment name	Reward strategies	α_{ac}^θ
<i>experiment_1</i>	Shaped rewards	$1e-6$
<i>experiment_2</i>	Shaped rewards, past rewards	$1e-6$
<i>experiment_3</i>	Shaped rewards, past rewards	$1e-5$

Table 4: PPO experiments’ setups

Experiment name	Reward strategies	α_{ppo}^θ
<i>experiment_4</i>	Shaped rewards, past rewards	$1e-6$
<i>experiment_5</i>	Shaped rewards, past rewards	$1e-5$
<i>experiment_6</i>	Shaped rewards, past rewards, delivery rewards	$1e-5$

5.4 Training results

In figure 1 we report the training curves for all our experiments; at each episode, we plot the **average** episodic reward achieved until that episode. With table 5 we show the average values over 10 episodes achieved by our agents after training, under a test setting. Looking at the training curve and the test results, for the actor-critic (*experiment_1*) we can see how shaped rewards are not enough to achieve a successful policy and the average reward actually falls down as the training goes. By rewarding the previous K actions that led to a reward, we noticed that our agents continued to improve their policy by achieving a higher cumulative rewards with *experiment_2*. Since learning was happening quite slowly, we tried to increase the learning rate for the policy network (*experiment_3*), with the results of approaching towards a suboptimal deterministic policy (the training curve reached a plateau and at test time we noticed our agents to play almost deterministically).

We tried to improve our solution by using the PPO algorithm; rewarding successful trajectories showed promising results, and we started from there. With *experiment_4* we noticed a slower increase in the average reward compared to actor-critic in *experiment_2*, but still in the promising direction. We tried to increase the learning rate of the policy network in *experiment_5*, noticing a quicker learning time at the expenses of an early plateau. Here, as shown in table 5, we noticed our agents to be able to prepare and pick up soups, without managing to deliver them. We concluded our experiments with *experiment_6*, also by rewarding the delivered soups with less than 3 onions. In this scenario our agents learned to prepare, pick up, and deliver the soups by achieving almost double the average return with respect to the previous experiment. Given the promising direction of *experiment_6*, we pushed the training forward to 5000 epochs, achieving an average sparse reward of 172.0 (8.6 soups with 3 onions per episode), as shown in the last row of table 5.

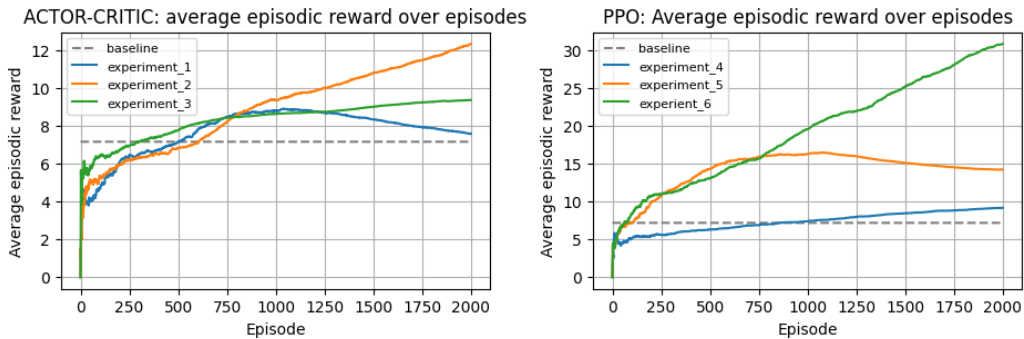


Figure 1: Average episodic reward (sparse + shaped) for actor-critic and PPO during the training process. The grey dashed line represents the random baseline.

Table 5: Average values achieved on 10 (test) episodes for each training setting

Training Epochs	Experiment Name	Rewards		Onions		Soups	
		Sparse	Shaped	Pickups	Potting	Pickups	Delivered
-	<i>baseline</i>	0.2	7.17	8.36	1.57	0.51	0.15
2000	<i>experiment_1</i>	0.0	8.7	13.4	1.6	4.0	0.0
2000	<i>experiment_2</i>	0.0	16.4	8.9	2.7	3.9	0.2
2000	<i>experiment_3</i>	0.0	10.2	4.2	3.0	0.0	0.0
2000	<i>experiment_4</i>	2.0	12.6	7.4	1.9	1.1	0.3
2000	<i>experiment_5</i>	0.0	20.6	10.2	4.5	1.6	0.0
2000	<i>experiment_6</i>	10.0	36.2	7.7	5.6	1.6	1.4
5000	<i>experiment_6</i>	172.0	325.8	25.5	28.5	8.8	8.7

6 Discussion

The reward strategies defined in 4.2 have shown to be the key learning ingredient; on the other hand our agents might have learned specific roles (e.g. chef vs waiter) as they have learned a specific sequence of successful action to perform and received a single reward associated to both the agents. As a matter of fact, in all our attempts we noticed a role specialization, but still showing coordination capabilities. Our implementation of PPO turned out to be the most effective in solving the cooperative environment overcooked; we suspect that the problem in the actor-critic implementation was in the weights’ update of the value function defined in (1). In other experiments not reported here, we managed to achieve results comparable to the ones of *experiment_6* by using actor-critic algorithm with deeper and wider networks in more than 10000 episodes. We found it impossible to tune the learning rates to speed up the training as the value function was not able to correctly estimate the value of the states, with the consequence of breaking the learning of the policy. Using the actor-critic with the MSE defined in (5) might deliver similar results obtained with *experiment_6*.

6.1 Future improvements

We could easily extend our solution to solve more layouts available in overcooked, thus by make our agents able to generalize over unseen layouts. State value functions to estimate the future expected value separately for each agent might improve cooperation (available in the implementation); more sophisticated advantage functions (e.g. GAE) might stabilize training and reduce the variance of the estimations; entropy loss might encourage exploration even after approaching the optimal policy (available in the implementation); replay buffers might avoid catastrophic forgetting and help the agents reuse previous successful experiences. Grid search on the algorithm’s hyper-parameters might significantly improve the results obtained in our experiments.

6.2 Statistical significance

All our experiments have been conducted by setting the *seed* = 42; given the short time at our disposal and the time needed to perform the training we couldn’t repeat the experiments using different seeds and average out the results. Nonetheless, this is a very important aspect that should always be considered when proving the effectiveness of new methods.

7 Conclusion

This report introduces two policy approximation methods to solve the cooperative benchmark environment overcooked. We describe and propose our actor-critic and proximal policy approximation implementations, together with reward strategies and hyperparameters setting to solve a single layout environment. We demonstrate our method to be a valuable approach and we suggest future possible implementation to solve multiple layouts for reaching generalization capabilities.

References

George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989. doi: 10.1007/BF02551274.

- Shengyi Huang, Rousslan Fernand Julien Dossa, Antonin Raffin, Anssi Kanervisto, and Weixun Wang. The 37 implementation details of proximal policy optimization. <https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/>, 2022.
- Human Compatible AI Lab. Overcooked ai environment. https://github.com/HumanCompatibleAI/overcooked_ai, 2020. Accessed: 2025-06-30.
- Nathan Lambert. Reinforcement learning from human feedback, 2025. URL <https://arxiv.org/abs/2504.12501>.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Belle-mare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- Mirco Musolesi. Policy gradient methods (part 2) — aas 2024/2025. https://www.mircomusolesi.org/courses/AAS24-25/AAS24-25_PolicyGradientMethods-SecondPart.pdf, 2025. Lecture slides, Accessed: 2025-06-30.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- David Silver, Aja Huang, Christopher Maddison, Arthur Guez, Laurent Sifre, George Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–489, 01 2016. doi: 10.1038/nature16961.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017. URL <https://arxiv.org/abs/1712.01815>.
- Elizaveta Tennant, Stephen Hailes, and Mirco Musolesi. Moral alignment for llm agents, 2025. URL <https://arxiv.org/abs/2410.01639>.
- Hao Wang, Yang Gao, and Xingguo Chen. RL-dot: A reinforcement learning npc team for playing domination games. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(1): 17–26, 2010. doi: 10.1109/TCIAIG.2009.2037972.
- Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3-4):279–292, 1992. doi: 10.1007/BF00992698.